

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Redes

Semestre II – 2026



Excelencia que trasciende

DEL VALLE
GRUPO EDUCATIVO

Proyecto 1

Uso de un protocolo existente

Avance 1

Integrantes:

Nájera Marakovits, Ingrid Nina Alessandra – 231088

1. Especificación de los servidores MCP desarrollados

Para este avance se implementó un chatbot de consola capaz de comunicarse con servidores MCP locales. El chatbot funciona como anfitrión y utiliza Gemini como modelo de lenguaje. La conexión con Gemini se realiza directamente mediante su API REST.

La comunicación con los servidores MCP se realiza mediante JSON-RPC 2.0. El cliente MCP fue implementado manualmente y se comunica con los servidores locales por medio de entrada y salida estándar. De esta forma se puede observar cada solicitud y cada respuesta que se intercambia durante la ejecución.

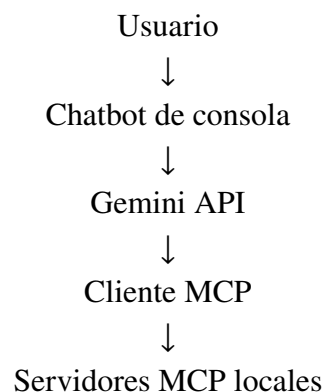
El proyecto se encuentra disponible en el siguiente repositorio:

<https://github.com/Ninaswiftie09/Redes/tree/Proyecto1>

1.1. Arquitectura utilizada

El chatbot inicia los servidores locales configurados y establece una comunicación independiente con cada uno. Durante el inicio se realiza la negociación del protocolo y luego se obtiene la lista de herramientas disponibles.

El flujo general utilizado es el siguiente:



El cliente utiliza la versión 2025-11-25 del protocolo MCP. Primero envía el mensaje `initialize`. Después de recibir una respuesta válida envía la notificación `notifications/initialized`. Finalmente utiliza `tools/list` para descubrir las herramientas y `tools/call` cuando necesita ejecutar alguna de ellas.

Cada mensaje enviado o recibido también se muestra en la terminal y se almacena en el archivo `logs/mcp.jsonl`. Esto permite revisar con claridad qué información se intercambió con cada servidor.

1.2. Servidores MCP locales

El proyecto tiene configurados tres servidores MCP locales.

Servidor	Tipo	Función
Filesystem	Oficial	Permite consultar, leer, crear y modificar archivos dentro del directorio autorizado del proyecto.
Git	Oficial	Permite realizar operaciones relacionadas con el repositorio Git local.
network_ops	Desarrollado	Proporciona herramientas sencillas para apoyar tareas de diagnóstico y análisis de redes.

El servidor Filesystem tiene acceso limitado al directorio del proyecto. Esto evita que pueda trabajar sobre cualquier ubicación del sistema.

El servidor Git se ejecuta sobre el repositorio actual y permite utilizar operaciones de Git desde el chatbot.

El servidor `network_ops` fue desarrollado específicamente para este proyecto y representa un caso de uso relacionado con operaciones de red.

1.3. Servidor Network Operations

El servidor desarrollado se llama `uvg-network-operations` y su versión actual es `0.1.0`.

Su propósito es apoyar a un equipo de soporte o de operaciones de red con tareas básicas que normalmente se necesitan al revisar un problema de conectividad.

El servidor se ejecuta localmente mediante Python y utiliza entrada y salida estándar como medio de comunicación. Por esta razón no utiliza un endpoint HTTP en este avance. Los mensajes JSON-RPC se escriben directamente en la entrada estándar del proceso y las respuestas se reciben mediante la salida estándar.

El servidor ofrece las siguientes operaciones MCP:

Método	Uso
<code>initialize</code>	Inicia la comunicación y devuelve la versión del protocolo, las capacidades y la información del servidor.
<code>ping</code>	Permite comprobar que el servidor continúa disponible.
<code>tools/list</code>	Devuelve la lista de herramientas disponibles junto con sus parámetros.
<code>tools/call</code>	Ejecuta una herramienta utilizando los argumentos enviados por el cliente.

1.4. Herramientas del servidor Network Operations

El servidor desarrollado cuenta con cuatro herramientas.

1.4.1. resolve_dns

Esta herramienta permite resolver un nombre de host y obtener las direcciones IP encontradas por el sistema.

Sus parámetros son:

Parámetro	Requerido	Descripción
hostname	Sí	Nombre del host o dirección que se desea resolver. El valor debe contener como máximo 253 caracteres.
ip_version	No	Permite indicar si se desean direcciones IPv4, IPv6 o cualquier versión. Si no se envía se utiliza any.

La respuesta contiene el nombre consultado, la versión solicitada y las direcciones encontradas.

Esta herramienta puede ser útil cuando un usuario puede acceder a una dirección IP pero tiene problemas al utilizar el nombre de un servicio.

1.4.2. check_tcp_connection

Esta herramienta intenta establecer una conexión TCP con un host y un puerto específico. Su objetivo es comprobar si un servicio puede ser alcanzado desde el equipo donde se ejecuta el servidor.

Sus parámetros son:

Parámetro	Requerido	Descripción
host	Sí	Nombre o dirección IP del equipo de destino.
port	Sí	Puerto TCP que se desea comprobar. Puede tener un valor entre 1 y 65535.
timeout_ms	No	Tiempo máximo de espera en milisegundos. Puede estar entre 100 y 10000. El valor utilizado por defecto es 2000.

La respuesta indica el host, el puerto, si fue posible establecer la conexión, el tiempo utilizado y cualquier error encontrado.

La herramienta solamente intenta una conexión al puerto solicitado. No realiza un escaneo de puertos y tampoco envía información de aplicación.

1.4.3. analyze_cidr

Esta herramienta permite analizar una dirección o red escrita con notación CIDR.

Recibe el siguiente parámetro:

Parámetro	Requerido	Descripción
cidr	Sí	Dirección IP acompañada de su longitud de prefijo.

El análisis se realiza de forma local y no genera tráfico de red.

Para IPv4 se obtiene información como la dirección de red, máscara, prefijo, broadcast, primera dirección utilizable, última dirección utilizable, cantidad total de direcciones y cantidad de direcciones utilizables.

También se identifica si la dirección pertenece a un rango privado.

Por ejemplo, al analizar la dirección 192.168.20.37/27, el servidor identifica que la red inicia en 192.168.20.32, el broadcast es 192.168.20.63 y existen 30 direcciones utilizables para hosts.

1.4.4. get_local_network_info

Esta herramienta obtiene información básica del equipo donde se está ejecutando el servidor.

No recibe parámetros.

La respuesta incluye:

- Nombre del equipo
- Nombre completo del host
- Direcciones IP encontradas por el sistema

Esta información puede utilizarse como un primer punto de referencia cuando se está revisando un problema de conectividad desde una estación de trabajo.

1.5. Formato de los mensajes

La comunicación entre el cliente y el servidor utiliza JSON-RPC 2.0.

Una solicitud para analizar una red se construye de la siguiente forma:

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "analyze_cidr",
    "arguments": {
      "cidr": "192.168.20.37/27"
    }
  }
}
```

```
}  
}
```

El servidor procesa la solicitud y devuelve una respuesta con el mismo identificador.

```
{  
  "jsonrpc": "2.0",  
  "id": 3,  
  "result": {  
    "content": [  
      {  
        "type": "text",  
        "text": "resultado del analisis"  
      }  
    ],  
    "isError": false  
  }  
}
```

El identificador permite relacionar cada respuesta con la solicitud que la generó.

Cuando existe un problema con los argumentos de una herramienta, el servidor devuelve el resultado indicando que ocurrió un error. Los errores propios del protocolo también utilizan los códigos definidos por JSON-RPC.

1.6. Integración con Gemini

El chatbot se conecta directamente a Gemini mediante el endpoint `generateContent` de su API REST.

Las herramientas descubiertas mediante MCP se transforman en declaraciones que Gemini puede utilizar. Cuando Gemini considera que necesita una herramienta, devuelve una solicitud de función.

Antes de ejecutar una herramienta solicitada por Gemini, el programa muestra en pantalla el nombre de la herramienta y sus argumentos. Luego solicita autorización al usuario. La herramienta solamente se ejecuta cuando la persona confirma la acción.

Después de recibir el resultado del servidor MCP, el chatbot lo devuelve a Gemini para que pueda generar una respuesta entendible para el usuario.

1.7. Contexto de conversación

El chatbot mantiene una lista con los mensajes enviados por el usuario y las respuestas generadas por Gemini durante la sesión actual.

Esto permite realizar preguntas relacionadas con mensajes anteriores sin repetir toda la información.

Por ejemplo, se puede preguntar primero quién fue Alan Turing y luego preguntar en qué año nació. La segunda pregunta conserva el contexto de la primera.

También existe el comando `/clear`, que permite borrar el contexto actual y comenzar una conversación nueva.

1.8. Registro de interacciones

Todas las comunicaciones MCP quedan registradas en el archivo:

`logs/mcp.jsonl`

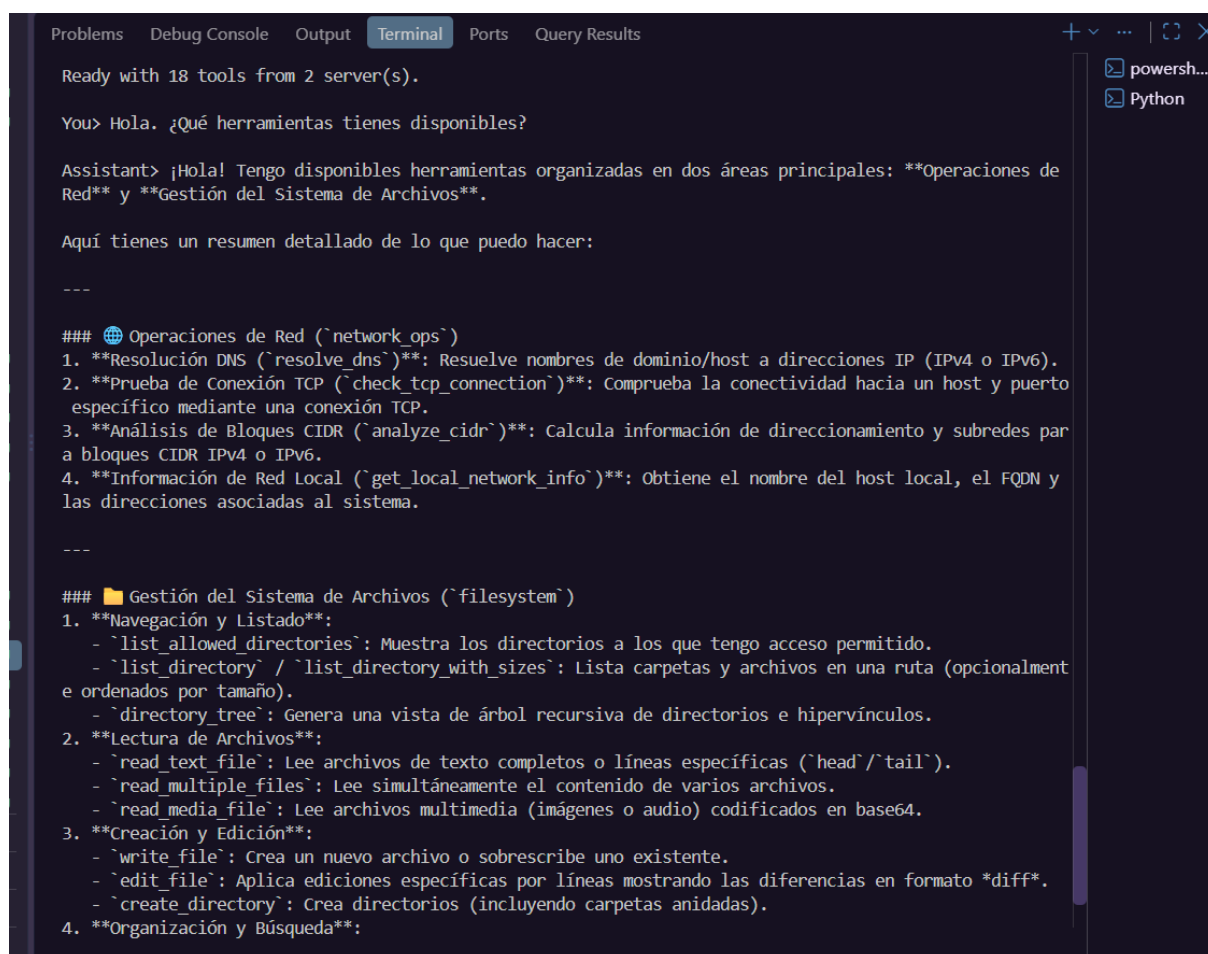
Cada registro contiene:

- Fecha y hora
- Nombre del servidor
- Dirección del mensaje
- Mensaje completo

Además de guardarse en el archivo, los mensajes se muestran en la consola con las etiquetas `SENT` y `RECEIVED`. Esto permite revisar directamente qué solicitud fue enviada y qué respuesta fue recibida.

1.9. Evidencia de funcionamiento

En la siguiente captura se observa el chatbot iniciado y la lista de herramientas disponibles. También se pueden observar herramientas del servidor `network_ops` y del servidor `Filesystem`.



```
Problems  Debug Console  Output  Terminal  Ports  Query Results

Ready with 18 tools from 2 server(s).

You> Hola. ¿Qué herramientas tienes disponibles?

Assistant> ¡Hola! Tengo disponibles herramientas organizadas en dos áreas principales: Operaciones de Red y Gestión del Sistema de Archivos.

Aquí tienes un resumen detallado de lo que puedo hacer:

---

### 🌐 Operaciones de Red (`network ops`)
1. Resolución DNS (`resolve_dns`): Resuelve nombres de dominio/host a direcciones IP (IPv4 o IPv6).
2. Prueba de Conexión TCP (`check_tcp_connection`): Comprueba la conectividad hacia un host y puerto específico mediante una conexión TCP.
3. Análisis de Bloques CIDR (`analyze_cidr`): Calcula información de direccionamiento y subredes para bloques CIDR IPv4 o IPv6.
4. Información de Red Local (`get_local_network_info`): Obtiene el nombre del host local, el FQDN y las direcciones asociadas al sistema.

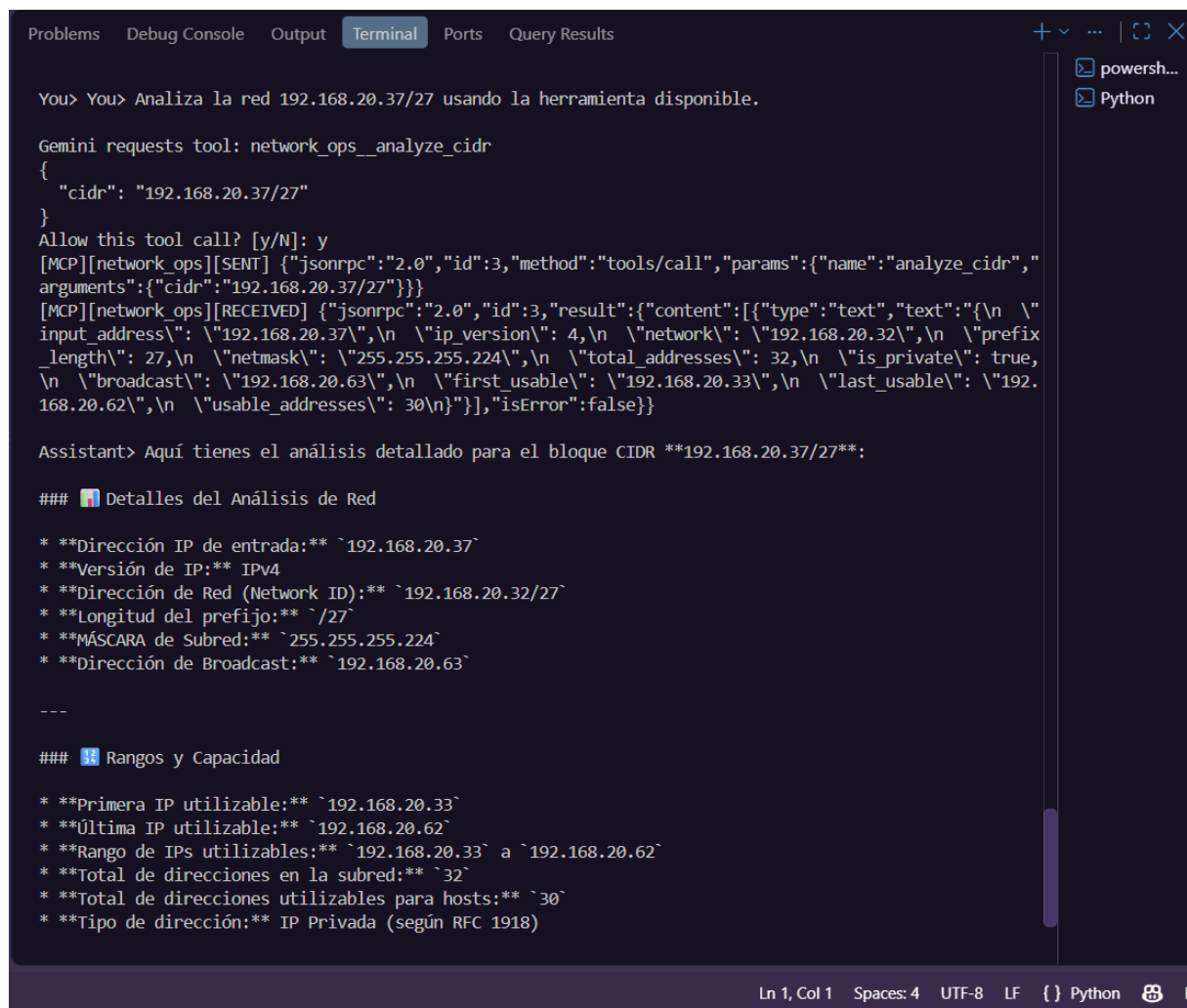
---

### 📁 Gestión del Sistema de Archivos (`filesystem`)
1. Navegación y Listado:
   - `list_allowed_directories`: Muestra los directorios a los que tengo acceso permitido.
   - `list_directory` / `list_directory_with_sizes`: Lista carpetas y archivos en una ruta (opcionalmente ordenados por tamaño).
   - `directory tree`: Genera una vista de árbol recursiva de directorios e hipervínculos.
2. Lectura de Archivos:
   - `read text file`: Lee archivos de texto completos o líneas específicas (`head`/`tail`).
   - `read_multiple_files`: Lee simultáneamente el contenido de varios archivos.
   - `read media file`: Lee archivos multimedia (imágenes o audio) codificados en base64.
3. Creación y Edición:
   - `write file`: Crea un nuevo archivo o sobrescribe uno existente.
   - `edit_file`: Aplica ediciones específicas por líneas mostrando las diferencias en formato `diff`.
   - `create_directory`: Crea directorios (incluyendo carpetas anidadas).
4. Organización y Búsqueda:
```

Figura 1: Herramientas disponibles en el chatbot

En la siguiente prueba se solicitó analizar la red 192.168.20.37/27. Gemini identificó que debía utilizar la herramienta `analyze_cidr` y pidió autorización antes de ejecutarla.

También se observan en la consola el mensaje JSON-RPC enviado al servidor y la respuesta recibida. Finalmente, Gemini presenta el resultado de una forma más sencilla para el usuario.



The screenshot shows a VS Code terminal window with a dark theme. The terminal displays a chatbot conversation. The user asks to analyze the network 192.168.20.37/27. The chatbot uses a tool named 'analyze_cidr' and returns a detailed JSON response. The chatbot then summarizes the analysis in Spanish, providing details about the IP address, network, and broadcast range, as well as the number of usable IP addresses and the type of network (private).

```
Problems  Debug Console  Output  Terminal  Ports  Query Results

You> You> Analiza la red 192.168.20.37/27 usando la herramienta disponible.

Gemini requests tool: network_ops__analyze_cidr
{
  "cidr": "192.168.20.37/27"
}
Allow this tool call? [y/N]: y
[MCP][network_ops][SENT] {"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"analyze_cidr","arguments":{"cidr":"192.168.20.37/27"}}}
[MCP][network_ops][RECEIVED] {"jsonrpc":"2.0","id":3,"result":{"content":[{"type":"text","text":{"\n \n input_address\\": \"192.168.20.37\\",\n \n \"ip_version\\": 4,\n \n \"network\\": \"192.168.20.32\\",\n \n \"prefix_length\\": 27,\n \n \"netmask\\": \"255.255.255.224\\",\n \n \"total_addresses\\": 32,\n \n \"is_private\\": true,\n \n \"broadcast\\": \"192.168.20.63\\",\n \n \"first_usable\\": \"192.168.20.33\\",\n \n \"last_usable\\": \"192.168.20.62\\",\n \n \"usable_addresses\\": 30\n}}],"isError":false}}

Assistant> Aquí tienes el análisis detallado para el bloque CIDR **192.168.20.37/27**:

### 📊 Detalles del Análisis de Red

* **Dirección IP de entrada:** `192.168.20.37`
* **Versión de IP:** IPv4
* **Dirección de Red (Network ID):** `192.168.20.32/27`
* **Longitud del prefijo:** `27`
* **MÁSCARA de Subred:** `255.255.255.224`
* **Dirección de Broadcast:** `192.168.20.63`

---

### 📏 Rangos y Capacidad

* **Primera IP utilizable:** `192.168.20.33`
* **Última IP utilizable:** `192.168.20.62`
* **Rango de IPs utilizables:** `192.168.20.33` a `192.168.20.62`
* **Total de direcciones en la subred:** `32`
* **Total de direcciones utilizables para hosts:** `30`
* **Tipo de dirección:** IP Privada (según RFC 1918)

Ln 1, Col 1  Spaces: 4  UTF-8  LF  {} Python  🐞  F
```

Figura 2: Uso de la herramienta analyze_cidr mediante MCP

Para comprobar el funcionamiento general del chatbot también se realizó una prueba de contexto. Primero se preguntó quién fue Alan Turing.

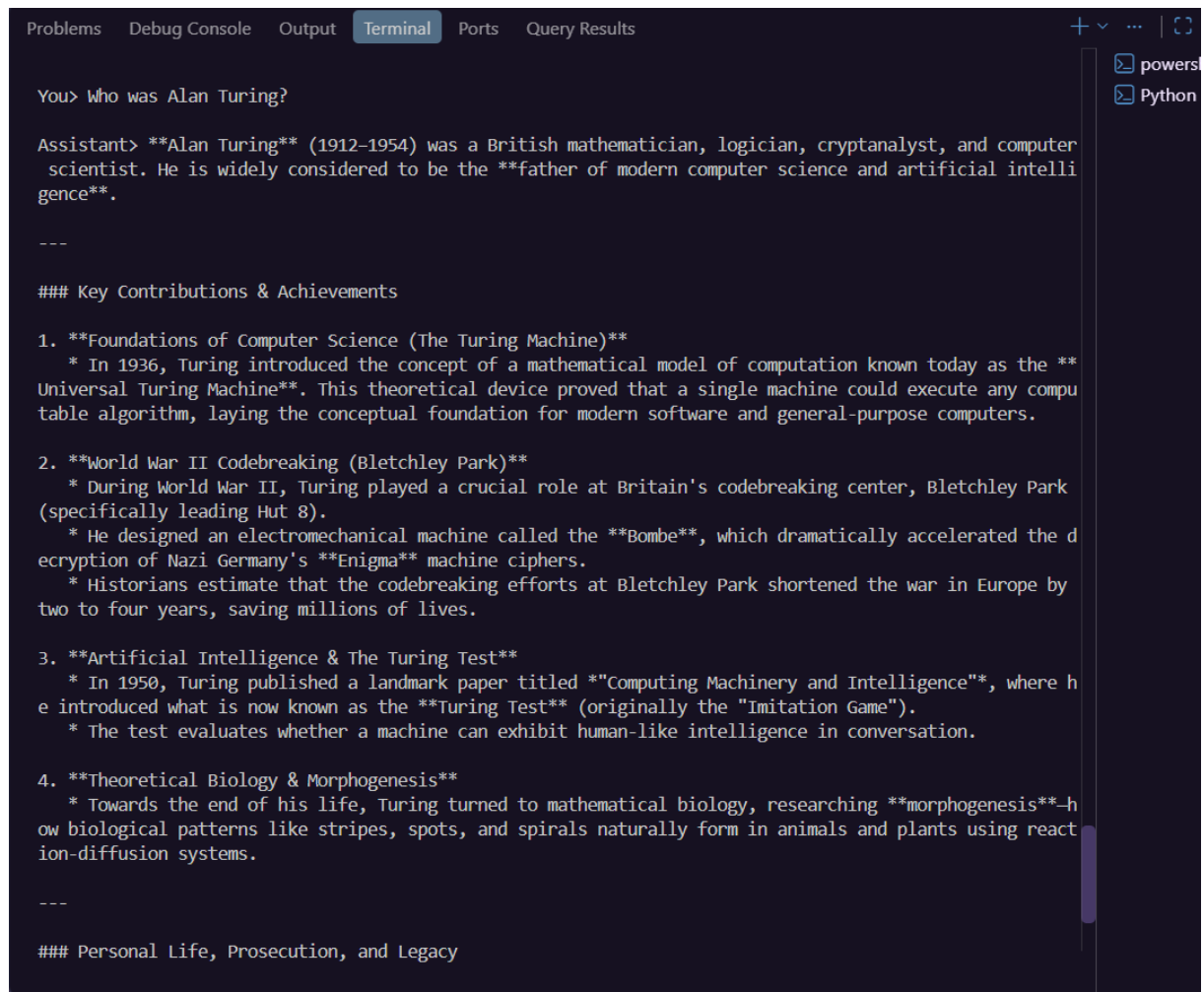
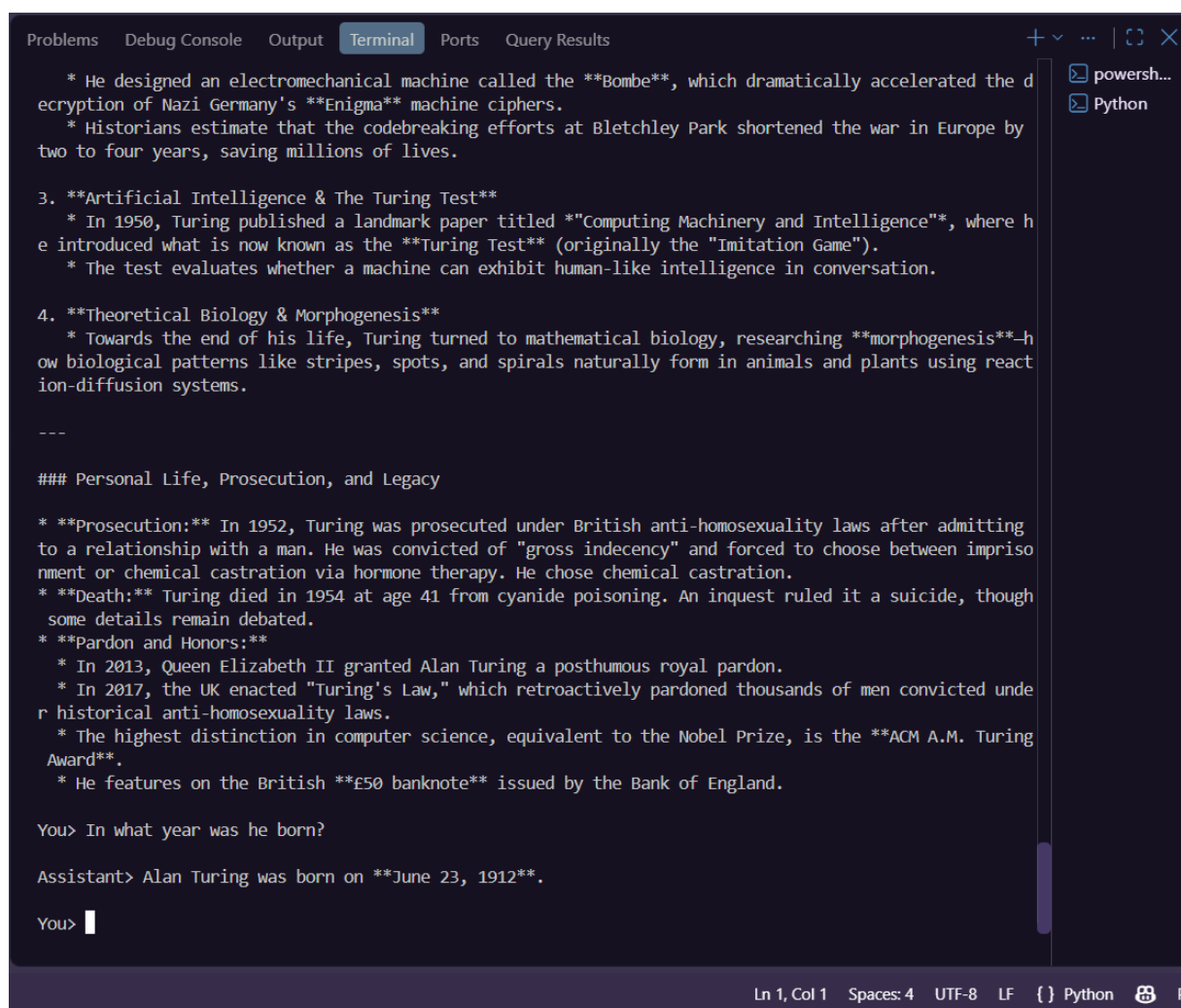
A screenshot of a terminal window with a dark background. The terminal has tabs at the top: 'Problems', 'Debug Console', 'Output', 'Terminal' (which is active), 'Ports', and 'Query Results'. On the right side, there are icons for 'powershell' and 'Python'. The text in the terminal shows a user asking 'Who was Alan Turing?' and an assistant responding with a detailed biography and a list of contributions. The assistant's response is formatted with bold text for names and sections, and italicized text for specific details. The list of contributions includes: 1. Foundations of Computer Science (The Turing Machine), 2. World War II Codebreaking (Bletchley Park), 3. Artificial Intelligence & The Turing Test, and 4. Theoretical Biology & Morphogenesis. The terminal also shows a separator '---' and a section header '### Personal Life, Prosecution, and Legacy'.

Figura 3: Primera pregunta de la prueba de contexto

Después se preguntó en qué año había nacido sin volver a escribir su nombre. El chatbot entendió que la pregunta seguía haciendo referencia a Alan Turing y respondió utilizando la información almacenada durante la misma sesión.



```
Problems  Debug Console  Output  Terminal  Ports  Query Results
+ v ... | [ ] X

* He designed an electromechanical machine called the Bombe, which dramatically accelerated the decryption of Nazi Germany's Enigma machine ciphers.
* Historians estimate that the codebreaking efforts at Bletchley Park shortened the war in Europe by two to four years, saving millions of lives.

3. Artificial Intelligence & The Turing Test
* In 1950, Turing published a landmark paper titled "Computing Machinery and Intelligence", where he introduced what is now known as the Turing Test (originally the "Imitation Game").
* The test evaluates whether a machine can exhibit human-like intelligence in conversation.

4. Theoretical Biology & Morphogenesis
* Towards the end of his life, Turing turned to mathematical biology, researching morphogenesis—how biological patterns like stripes, spots, and spirals naturally form in animals and plants using reaction-diffusion systems.

---

### Personal Life, Prosecution, and Legacy

* Prosecution: In 1952, Turing was prosecuted under British anti-homosexuality laws after admitting to a relationship with a man. He was convicted of "gross indecency" and forced to choose between imprisonment or chemical castration via hormone therapy. He chose chemical castration.
* Death: Turing died in 1954 at age 41 from cyanide poisoning. An inquest ruled it a suicide, though some details remain debated.
* Pardon and Honors:
  * In 2013, Queen Elizabeth II granted Alan Turing a posthumous royal pardon.
  * In 2017, the UK enacted "Turing's Law," which retroactively pardoned thousands of men convicted under historical anti-homosexuality laws.
  * The highest distinction in computer science, equivalent to the Nobel Prize, is the ACM A.M. Turing Award.
  * He features on the British £50 banknote issued by the Bank of England.

You> In what year was he born?

Assistant> Alan Turing was born on June 23, 1912.

You> 
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF {} Python [] P

Figura 4: Segunda pregunta utilizando el contexto de la conversación

1.10. Pruebas realizadas

El proyecto incluye pruebas automáticas para validar las partes principales de la implementación.

Se verificó el manejo de contexto del chatbot, el uso de herramientas desde Gemini, la inicialización del cliente MCP, el descubrimiento de herramientas, la ejecución del servidor local, el análisis CIDR, el manejo de argumentos incorrectos y el comportamiento de las notificaciones.

Al ejecutar las pruebas se obtuvieron ocho pruebas completadas correctamente.

Esto permitió comprobar que el servidor local puede completar el flujo de inicialización, listar sus cuatro herramientas y ejecutar solicitudes mediante JSON-RPC.

2. Conclusiones y comentario sobre el proyecto

Este avance permitió entender mejor cómo un modelo de lenguaje puede trabajar con herramientas externas. Antes de realizar el proyecto podía parecer que el modelo era el que ejecutaba

directamente las acciones, pero en realidad el chatbot se encarga de conectar al modelo con los servidores y de enviar las solicitudes necesarias.

También fue útil implementar JSON-RPC de forma manual porque permitió ver con más claridad qué información se envía entre el cliente y el servidor. Al mostrar las solicitudes y respuestas en la consola es más fácil entender la función del identificador, los métodos, los parámetros y los resultados.

La creación del servidor de operaciones de red permitió relacionar MCP con un caso más cercano al curso. Las herramientas creadas pueden servir como apoyo para revisar problemas básicos de conectividad, resolver nombres, comprobar conexiones TCP, analizar redes CIDR y conocer información del equipo local.

Una parte importante del proyecto fue mantener separadas las responsabilidades. Gemini decide cuándo una herramienta puede ser útil, el chatbot administra el contexto y la comunicación, el cliente MCP envía los mensajes y cada servidor se encarga de ejecutar sus propias herramientas.

También considero importante que el usuario tenga que autorizar las herramientas antes de que sean ejecutadas. Esto permite ver exactamente qué acción quiere realizar el modelo y qué parámetros desea enviar antes de permitirla.

Las pruebas de contexto mostraron que el chatbot puede mantener una conversación relacionada durante una misma sesión. Las pruebas del servidor también permitieron comprobar que la comunicación MCP funciona y que los resultados llegan nuevamente hasta Gemini.

En general, este primer avance ayudó a comprender mejor la arquitectura de MCP y la forma en que se puede conectar un modelo de lenguaje con herramientas reales. Para la siguiente entrega será necesario trasladar el servidor desarrollado a un entorno remoto y analizar la comunicación de red con Wireshark.